Build What Pays

Michael Colenso

A Solo Builder's Field Manual for Finding Ideas That Pay

April 03, 2026

Contents

1. Why This Book 4
Foundations 5
Introduction 5
1.1. The Idea Trap 6
1.1.1. Novelty Bias 7
1.1.2. Proximity Bias 8
1.1.3. Tool-First Thinking 8
1.1.4. Compliment Confusion 9
1.1.5. Scope Intoxication 10
1.2. The Money-First Lens 11
1.2.1. Current Spending 11
1.2.2. Costly Mistakes 11
1.2.3. Time-Sensitive Decisions 11
1.3. The Six Filters 12
1.3.1. Buyer Clarity 13
1.3.2. Pain Severity 13
1.3.3. Monetization Clarity 13
1.3.4. Build Feasibility 14
1.3.5. Defensibility 14
1.3.6. Distribution Realism 14
1.4. Three Discovery Engines 14
1.4.1. Engine 1: Money Trails in Communities 14
1.4.2. Engine 2: Public-Data Arbitrage 14
1.4.3. Engine 3: Workflow Compression 15
1.5. Kill Signals 15
1.5.1. The buyer is blurry 16
1.5.2. It smells like a feature, not a product 16
1.5.3. The path to money is hand-wavy 16
1.5.4. The pain is polite, not painful 16
1.5.5. Distribution is wishful 16
1.5.6. The wedge is too broad 16
1.5.7. You cannot reach the buyer 17
1.5.8. The product needs too much before it can prove anything 17
1.5.9. It is legally contaminated 17
1.5.10. You are excited mainly because the tooling is fun . . . 17
1.6. From Signal to Wedge 17
1.6.1. The Wedge Test 18
Validation and Execution 19
1.7. The 30-Day Validation Sprint 19
1.7.1. Week 1: Verify the Pain 19
1.7.2. Week 2: Test the Offer 19
1.7.3. Week 3: Deliver Manually 19

1.7.4. Week 4: Decide Brutally 20
1.8. First Customer Before Full Product 20
1.9. Framework in Action 20
1.9.1. 1. SauceStack 21
1.9.2. 2. FlipTrack 21
1.9.3. 3. Building Permit Cross-Sell Intelligence 21
1.9.4. 4. Federal Contracting Opportunity Engine 22
1.9.5. 5. CraftLedger 22
1.10. What to Build Next 22
1.11. What Comes After the Wedge 22
Conclusion 23
2. Appendices 25
I Pattern Library 25
2.1.0.1. HiveStack 25
2.1.0.2. Liquor License Opening Alerts 25
2.1.0.3. CFPB Complaint Intelligence 25
2.1.0.4. Restaurant Inspection Crisis Alerts 25
2.1.0.5. Contractor License Violation Monitor 25
2.1.0.6. Small-Batch Production Tracker 25
2.1.0.7. UCC Filing Lead Intelligence 25
2.1.0.8. Compliance Launch Planner 25
II Opportunity Scorecard 26
III Opportunity Worksheet 26

1. Why This Book

I did not build this framework because I wanted a cleaner way to brainstorm.

I built it because I got tired of watching smart builders waste time on weak bets.

My working life has been shaped by operations, sequencing, constraints, and consequence. I spent more than twenty years in construction project management, where vague thinking gets expensive fast. On a real project, you do not get points for interesting ideas. You get measured on whether the thing works, whether the timing holds, whether the budget survives contact with reality, and whether the risks were visible early enough to matter. That operating instinct came with me into software, automation, data products, and AI-assisted builds.

The pattern showed up immediately.

Builders were not short on ideas. They were drowning in them. Every trend produced another wave of "maybe this could be a startup" energy. Every new model or tool created another excuse to go hunting for a problem after the solution already felt exciting. Every founder thread became another reminder that the market rewards judgment more than ideation, but almost nobody was building systems for judgment.

So I started scoring opportunities harder.

Not in a theoretical way. In a practical, repetitive, operator way. Who is the buyer? How painful is the problem? Is money already moving? Is the trigger obvious? Can the first version ship without drowning in complexity? Is there a believable path to distribution, or am I lying to myself because the build sounds fun?

I have evaluated more than two hundred opportunities this way. Most of them failed the filters.

That is the point.

A good framework should reject more ideas than it advances. If it blesses everything, it is not a framework. It is a mood board.

This book came out of those repetitions. It is a compressed system built from seeing the same failure modes over and over: novelty bias, vague buyers, no trigger, no path to money, too much software before proof, and founders mistaking their own excitement for market reality.

If you use this book well, it will probably save you from building a few things you currently like. Good. That is cheaper than building them.

Foundations

This part builds the evaluation system: how to spot stronger opportunities, reject weak ones, and narrow signal into a real wedge.

Introduction

Most builders do not fail because they cannot execute. They fail because they choose weak ideas, validate too late, and confuse interest with demand.

The most common failure mode is not execution failure. It is selection failure.

They mistake motion for evidence. They generate long lists of plausible products, follow whatever is trending, or fall in love with a tool before they have earned a problem. Then they spend weeks or months building something that never had enough pain, urgency, or spending behind it to matter.

That is the trap.

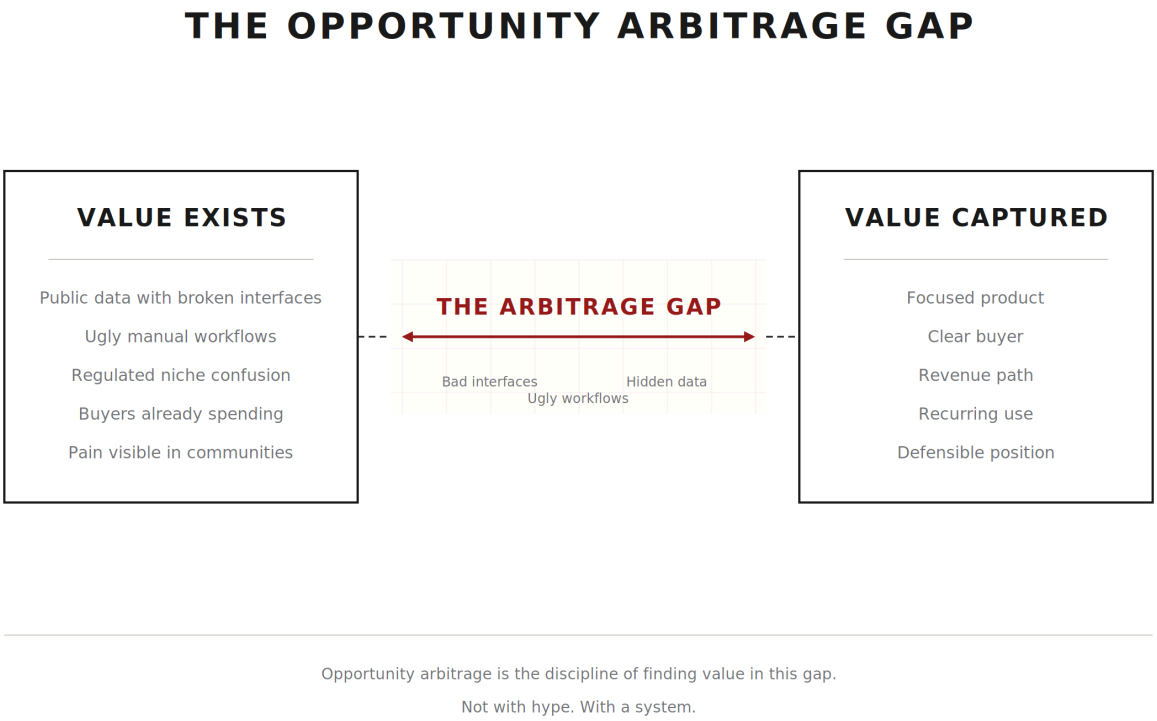
The market leaks signal everywhere. Operators complain about ugly workflows in forums. Small businesses pay monthly for software they resent. Public data sits in broken interfaces with zero packaging and obvious commercial value. Buyers reveal intent through permits, licenses, inspections, filings, violations, complaints, recalls, and other operational signals long before most founders notice.

Opportunity arbitrage is the discipline of finding value in that gap.

Opportunity arbitrage: finding markets where value already exists but is poorly surfaced, poorly packaged, or poorly served.

Not with hype. Not with generic "AI startup ideas." Not with another brainstorm dump.

With a system.



A strong solo-builder opportunity usually has some combination of these traits:

- a clear buyer
- a painful, repeated problem
- visible money already changing hands
- a narrow version that can ship fast
- a believable path to first revenue
- enough defensibility to matter, without so much complexity that the build collapses under its own ambition

The goal is not to find the most glamorous idea. The goal is to find the strongest wedge.

Wedge: the narrowest, most specific version of a product that can prove demand. Not the whole vision. The entry point.

This book will help you do three things: find higher-signal opportunities, kill weak ideas faster, and validate before you overbuild.

If this book works, it should make you more ruthless. Ruthless about what counts as evidence. Ruthless about what gets cut. Ruthless about what deserves your next 30 days.

1.1. The Idea Trap

Most builders think their job starts with ideas. It does not. It starts with filters.

Good founders are not idea generators. They are idea editors.

An unfiltered idea pipeline produces junk. You can make almost anything sound plausible if you stay abstract enough: a marketplace for X, an AI assistant for Y, a dashboard for Z. Plausibility is cheap. The market does not pay for plausibility.

The early game is not creativity. It is discrimination. You are trying to separate problems that feel interesting from problems that create enough pain, urgency, and budget to support a business. That means asking blunt questions early: Who has the problem? How do they describe it now? What does it already cost them? Who can authorize spend? What narrow wedge gets you into the workflow before the grand vision shows up?

Most weak ideas survive because they are protected by vagueness. Nobody has to name the buyer. Nobody has to define the pain in operational terms. Nobody has to say what the first paid offer is, how the product wedges into an existing workflow, or why this beats the status quo enough to trigger action. As long as the idea stays abstract, it can keep sounding smart.

Weak ideas usually survive for the same reasons.

1.1.1. Novelty Bias

Builders chase what feels new, not what is painful. A clever product gets more attention in founder circles than a boring product that solves a recurring operational headache.

This is one of the cleanest ways to waste six months. Novelty creates energy. It gives the builder a story to tell: nobody is doing this, the tech just changed, the timing feels right. But buyers rarely wake up wanting novelty. They wake up wanting fewer errors, faster throughput, lower labor, less rework, better visibility, fewer missed deadlines, or more revenue from the same base of demand.

That is why dull markets often pay better than exciting ones. If a dispatch manager loses two hours a day to a broken handoff, or a PM burns margin because job-cost data lands late, they do not care whether the fix is glamorous. They care whether it removes drag from a workflow they already own. Pain with budget beats novelty with applause.

Novelty bias also makes founders overrate the competitive advantage of being early. Being early to a weak demand curve is not strategy. It is volunteering to educate the market for free. If customers are not already trying to solve the problem, already spending on adjacent fixes, or already complaining in ways that map to a budget line, then being first just means being unsupported.

The better question is not “Is this new?” It is “Is this expensive enough to matter?” A good wedge does not need to sound futuristic. It needs to

attach to a recurring cost center, a known headache, and a buyer who can say yes without rewriting the org chart.

1.1.2. Proximity Bias

Builders overvalue problems they personally find interesting, even when the buyer is unclear or the budget is weak.

Proximity bias shows up when the founder confuses personal familiarity with commercial validity. You touched a problem, so you assume the market is there. You find a workflow annoying, so you assume people will pay to fix it. Sometimes that is true. Often it is not. Being close to a problem can help you see nuance, but it can also make you blind to how little the buyer actually cares.

A problem can be real and still not be a business. Maybe it is annoying but infrequent. Maybe the pain sits with a user who cannot authorize spend. Maybe the company tolerates the mess because switching costs are higher than the drag. Maybe the workaround is ugly but good enough. Founders lose a lot of time building for these kinds of half-problems because they can vividly feel them themselves.

This gets worse when the builder is not explicit about whose pain matters. The end user may hate the workflow while the buyer only cares about compliance. The operator may want speed while the owner wants fewer write-offs. The team may praise the idea while nobody in finance sees a reason to budget for it. If you cannot distinguish user pain from buyer pain, you will mistake interest for demand.

Proximity helps when it gives you language, access, and credibility. It hurts when it becomes a substitute for validation. The operator move is to use your insider view to sharpen the wedge, not to skip the proof. Name the buyer. Quantify the pain. Find the existing budget. If you cannot do that, your closeness is probably camouflage.

1.1.3. Tool-First Thinking

AI, scraping, automation, agents, and workflows become excuses to go looking for a problem after the solution already feels exciting.

If the best argument for the idea is that the stack is interesting, the market is probably not the real driver.

Tool-first thinking usually sounds sophisticated because the tools are real and the capabilities are moving fast. The mistake is not liking the tools. The mistake is letting technical possibility lead commercial logic. Once the builder gets excited about what a stack can do, the market research quietly becomes a scavenger hunt for a problem that justifies the build.

That is backward. Buyers do not purchase “AI automation” in the abstract. They purchase fewer manual touches, faster turnaround, better conversion, less admin labor, cleaner reporting, tighter follow-up, or

some other concrete operational outcome. The tool is implementation detail. The buyer is paying to remove friction or create money.

This is why many tool-first ideas collapse under basic questioning. What job is painful enough that someone wants this fixed now? Who owns that job? What metric moves if this works? What current workaround are they using? What would they stop paying for, reduce, or avoid by adopting this? If the answers stay fuzzy while the technical architecture gets more detailed, the idea is upside down.

There is nothing wrong with building around a capability shift. New tools do create openings. But the wedge still has to be anchored in a specific painful workflow with a clear buyer and a believable before-and-after. "We can build an agent that does X" is not the thesis. "Collections teams are bleeding cash because follow-up is inconsistent, and this replaces a measurable chunk of missed recovery" is a thesis.

1.1.4. Compliment Confusion

People say, "That's cool," or, "I'd use that," and founders treat it like validation. It is not. Real validation has cost, effort, urgency, or money behind it.

Compliments are socially cheap. People like encouraging builders. They like sounding supportive, smart, and early. So they say the idea is interesting, the demo is slick, or they would totally use it. None of that means they will change behavior, make time, take risk, or open a budget.

The easiest trap is mistaking positive language for demand signal. Someone says, "We need something like this," but they will not take a second call, introduce the buyer, send sample data, or explain their current process in detail. That is not validation. That is politeness. Real demand starts showing up when the prospect invests something scarce: time, access, reputation, workflow disruption, or money.

Good signals have weight. A prospect gives you exports from their current system. They pull in the ops lead. They ask how soon you can pilot. They ask security questions. They want pricing. They agree to manual onboarding because the problem is painful enough right now. Those signals are not perfect, but they are expensive enough to matter.

This matters because compliment confusion creates fake momentum. The founder hears enough positive noise to keep building, but not enough committed behavior to prove a wedge. The operator standard is simple: treat enthusiasm as a lead, not a verdict. Until someone pays, commits, introduces, or changes behavior, you have interest at best.

1.1.5. Scope Intoxication

The idea sounds big enough to matter, so the founder expands it before proving the smallest commercial wedge.

Scope intoxication is what happens when the founder falls in love with the full map before winning the first block. The product becomes a platform, then a network, then an operating system for an industry. Every adjacent workflow starts to look strategically important. In reality, this is usually avoidance. Expansion feels like ambition, but very often it is a way to postpone the harder question of whether any narrow slice is valuable enough to buy.

Markets are not won by describing total potential. They are entered through a wedge. A wedge is not the whole vision. It is the smallest painful, purchasable, adoptable slice of the vision. It solves one costly problem for one buyer in one workflow strongly enough to earn trust, usage, and money. Until that exists, broad scope is mostly decorative.

Wide scope also damages validation quality. The more surfaces you include, the harder it is to know what people are actually responding to. Are they interested in the analytics, the automation, the data layer, or the promise that it will someday connect everything? If the offer is too broad, feedback becomes impossible to interpret and implementation becomes slow enough to hide whether demand was ever there.

The commercial move is almost always to cut harder. Pick the buyer with money. Pick the workflow with recurring pain. Pick the narrowest outcome you can deliver fast. If the wedge works, expansion gets easier because you now have a live account, real language, and a reason to exist. If the wedge does not work, the grand vision was never helping you anyway.

THE FIVE FAILURE MODES

Why weak ideas survive long enough to waste months

NOVELTY BIAS
Chasing what feels new instead of what is painful

PROXIMITY BIAS
Overvaluing problems you find interesting when the buyer is unclear

TOOL-FIRST THINKING
Building the solution before earning the problem

COMPLIMENT CONFUSION
Treating "that's cool" as validation when real validation has money behind it

SCOPE INTOXICATION
Expanding the idea before proving the smallest commercial wedge

The idea is not the starting point. The filter is.

CHAPTER 1: THE IDEA TRAP

The result is predictable: months of work, weak demand, and a stack of explanations for why the market just was not ready.

Good founders are not just idea generators. They are idea editors. They know how to cut.

1.2. The Money-First Lens

A money-first lens does not mean greed-first. It means evidence-first.

Money-first ≠ greed-first. It means: where is evidence of spending, loss, or time-burn already visible?

The strongest early signal is not, "People care about this topic." It is, "People are already paying, losing money, or burning serious time because this problem is unresolved."

There are three especially strong signals.

1.2.1. Current Spending

If people already pay for an ugly tool, a manual service, a consultant, an attorney, or a spreadsheet-heavy process, there is usually room to build something sharper.

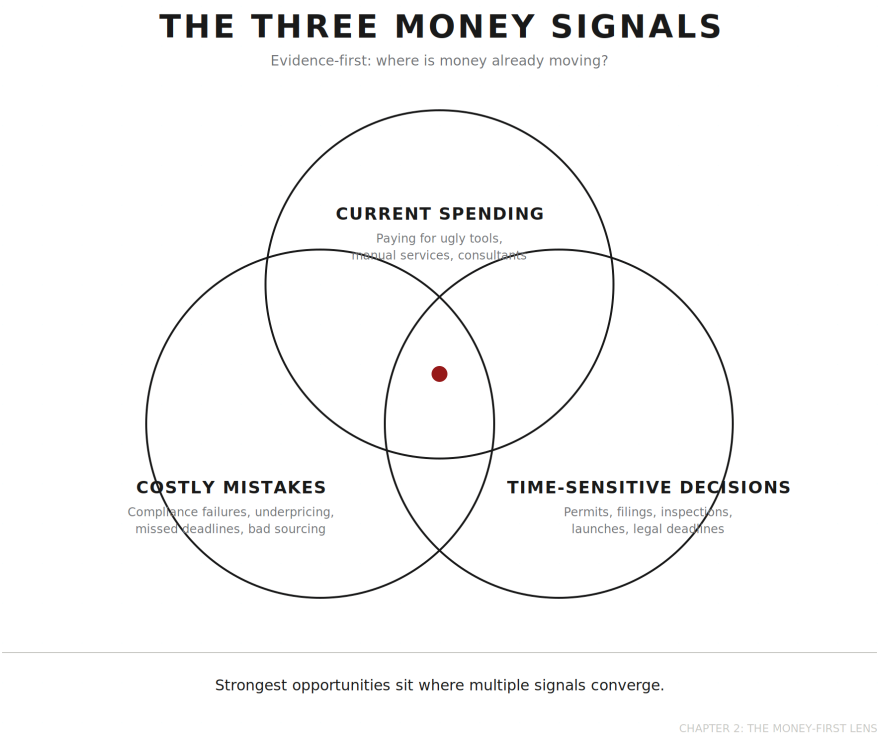
1.2.2. Costly Mistakes

Markets with repeated errors, compliance failures, underpricing, missed deadlines, bad sourcing, or weak tracking create stronger purchase intent than markets full of casual wish lists.

1.2.3. Time-Sensitive Decisions

Urgency is the best sales accelerant. Permits, inspections, and compliance deadlines create it naturally.

If a buyer needs the answer now because of a permit, launch, filing, inspection, deadline, or legal risk, urgency compresses the sales cycle.



A money-first builder asks: What are they already paying for? What mistakes are currently costing them money? What ugly process are they tolerating because nothing better exists? Can software, AI, or automation collapse this pain enough to matter?

That is why small, specific markets so often beat broad inspirational categories. A reseller trying to survive tax season, a contractor trying to catch permits before competitors, or a food founder trying to launch legally has a stronger buying trigger than someone vaguely asking for better productivity.

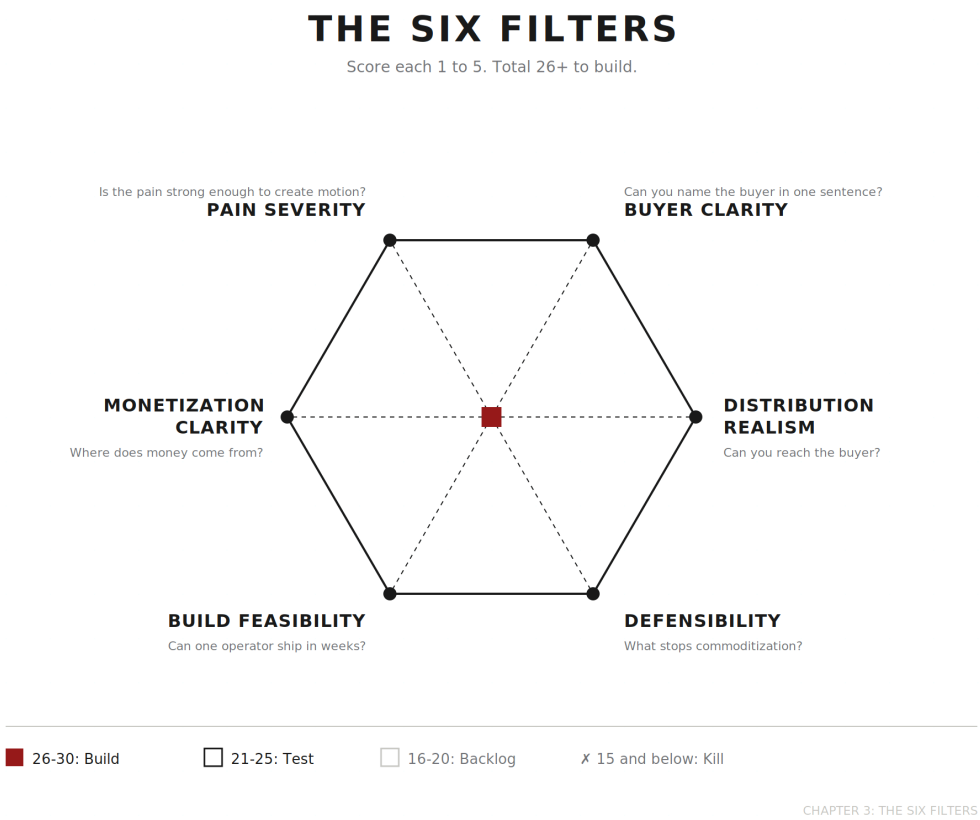
Specific pain creates specific products.

1.3. The Six Filters

Most idea evaluation is too soft. Founders say things like "big market," "strong trend," or "interesting space." None of that is enough.

Use these six filters on every opportunity before you let yourself like it.

Score each filter 1–5. Total of 26+ = build candidate. 15 or below = kill.



1.3.1. Buyer Clarity

Can you name the buyer in one sentence?

Test: "first-time hot sauce entrepreneur trying to sell legally" passes. "Small businesses" does not.

Not "small businesses." Not "creators." Not "anyone who needs help with X."

A useful buyer definition sounds like this: first-time hot sauce entrepreneur trying to sell legally; part-time reseller with tax-season spreadsheet chaos; leather crafter underpricing custom commissions; roofer who wants permit-triggered sales alerts in one county.

If the buyer is fuzzy, the product will be fuzzy.

1.3.2. Pain Severity

Is the problem painful enough to create motion?

Strong pain usually costs money, creates legal or compliance risk, wastes significant time, causes repeated operational mistakes, blocks revenue, or creates public embarrassment. A mild inconvenience rarely converts. A recurring operational wound often does.

1.3.3. Monetization Clarity

How does this make money quickly and believably?

If the money story requires a paragraph of explanation, the idea is weaker than it feels.

There should be a visible path to subscription, lead generation, affiliate revenue, premium alerts, service revenue before software, or a narrow B2B tool with obvious ROI.

1.3.4. Build Feasibility

Can one operator get to proof without dying in complexity? The question is not whether the idea can be built eventually. Nearly anything can. The question is whether the smallest wedge can ship in weeks, not quarters.

1.3.5. Defensibility

If this works, what stops it from becoming a commodity? Early defensibility does not need to mean a giant moat. It means there is something that gets harder to copy over time: curated data, workflow fit, niche trust, domain-specific scoring, or recurring operational use.

1.3.6. Distribution Realism

Do you have a believable path to the buyer? A strong idea with weak access can still be a bad bet. If the distribution thesis is basically hope, the opportunity is not ready.

A strong idea with weak distribution is still a bad bet. Hope is not a channel.

The point is not false precision. The point is forced judgment.

1.4. Three Discovery Engines

Good opportunities tend to emerge from the same few mechanics. Once you understand the mechanics, you stop waiting for inspiration.

Stop asking for ideas. Start running engines.

1.4.1. Engine 1: Money Trails in Communities

Look where people openly discuss what they pay for, what tools they hate, what mistakes cost them money, what manual work they are tired of doing, and what services they wish were better or cheaper.

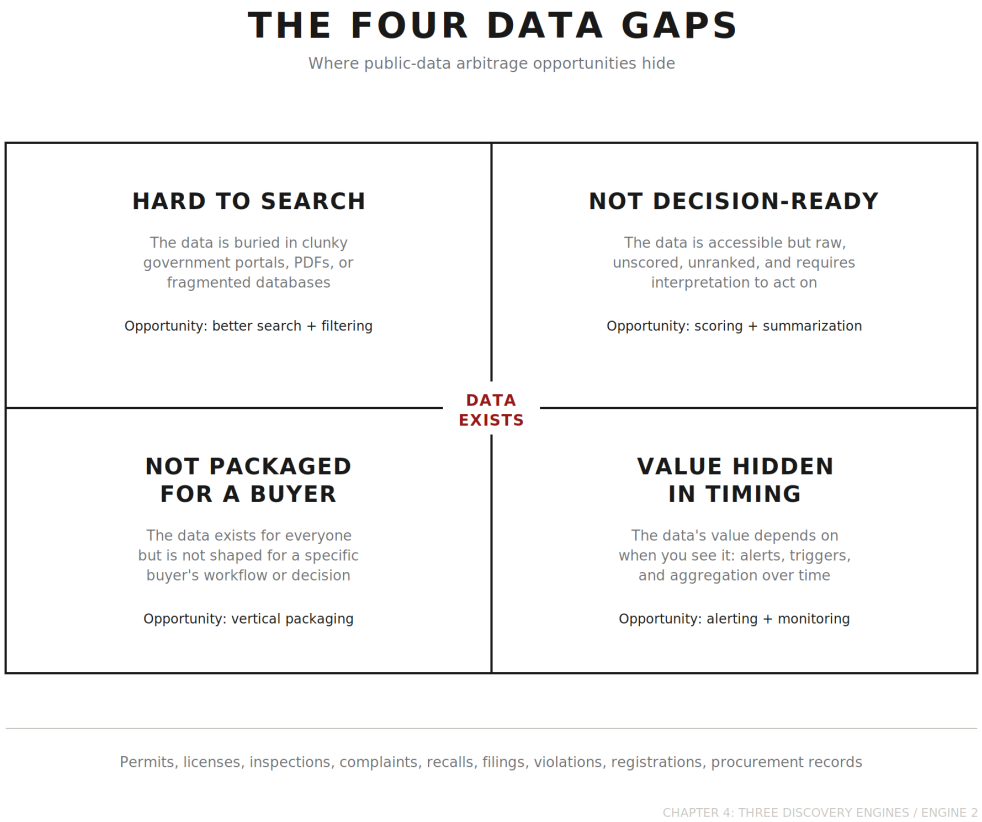
Good hunting grounds include subreddits, niche forums, Facebook groups, review sites for ugly vertical software, and practitioner-heavy YouTube comment sections.

You are looking for evidence, not vibes.

1.4.2. Engine 2: Public-Data Arbitrage

This engine works when valuable data exists but the interface is broken, the SEO is weak, or the commercial use case is hidden.

Look for: permits, licenses, inspections, complaints, recalls, filings, violations, registrations, procurement records.



1.4.3. Engine 3: Workflow Compression

Some markets still run on spreadsheets, copy-paste lookups, scattered vendor tabs, ad hoc checklists, inbox-and-memory project management, and brittle manual processes.

A workflow-compression product does not need to be revolutionary. It needs to reduce steps, reduce mistakes, and save time.

That is fertile ground.

Most strong solo-builder products come from one engine or a combination of two. The point is to stop asking for ideas and start running engines.

1.5. Kill Signals

A lot of progress comes from saying no faster.

Bad ideas do not fail quietly. They leak weakness early.

The strongest founders are not the ones who discover the most opportunities. They are the ones who kill weak ones fastest.

KILL SIGNAL CHECKLIST

Bad ideas leak weakness early. Check before you commit.

- ☐ **BUYER IS BLURRY**
  Cannot say exactly who this is for in one sentence
- ☐ **FEATURE, NOT PRODUCT**
  No standalone buying trigger; only works as an add-on
- ☐ **PATH TO MONEY IS HAND-WAVY**
  "Maybe ads," "maybe marketplace later" are not monetization plans
- ☐ **PAIN IS POLITE, NOT PAINFUL**
  No urgency, cost, embarrassment, or operational consequence
- ☐ **DISTRIBUTION IS WISHFUL**
  No believable path to the buyer beyond hope
- ☐ **WEDGE IS TOO BROAD**
  First version tries to serve the whole category
- ☐ **CANNOT REACH THE BUYER**
  Trust barriers, procurement cycles, or domain constraints block access
- ☐ **NEEDS TOO MUCH TO PROVE ANYTHING**
  Requires heavy infrastructure, integrations, or polished UX before validation
- ☐ **LEGALLY CONTAMINATED**
  Path depends on legal gray zones or high-liability behavior
- ☐ **EXCITED ABOUT TOOLING, NOT MARKET**
  The stack is the real draw, not the buyer's pain

Two or more checked = KILL or reshape the wedge

CHAPTER 5: KILL SIGNALS

1.5.1. The buyer is blurry

If you cannot say exactly who this is for, the problem is not ready.

1.5.2. It smells like a feature, not a product

If it only works as an add-on to a larger system and has no standalone buying trigger, it may not deserve to exist on its own.

1.5.3. The path to money is hand-wavy

“Maybe ads,” “maybe marketplace later,” and “maybe enterprise eventually” are not real monetization plans.

1.5.4. The pain is polite, not painful

If there is no urgency, cost, embarrassment, or operational consequence, move on.

1.5.5. Distribution is wishful

If the distribution thesis is mostly hope, the opportunity is weaker than it looks.

1.5.6. The wedge is too broad

If the first version tries to serve everyone in a category, you are not choosing a wedge. You are avoiding the discipline of choosing.

Two or more kill signals checked = kill or reshape the wedge.

1.5.7. You cannot reach the buyer

A problem can be real and still be a bad bet if the buyer is behind trust barriers, long procurement cycles, or domain constraints you do not want to fight.

1.5.8. The product needs too much before it can prove anything

If the wedge requires many integrations, polished UX, or heavy infrastructure before it can validate, that is a warning.

1.5.9. It is legally contaminated

Some opportunities are real but not worth the exposure.

1.5.10. You are excited mainly because the tooling is fun

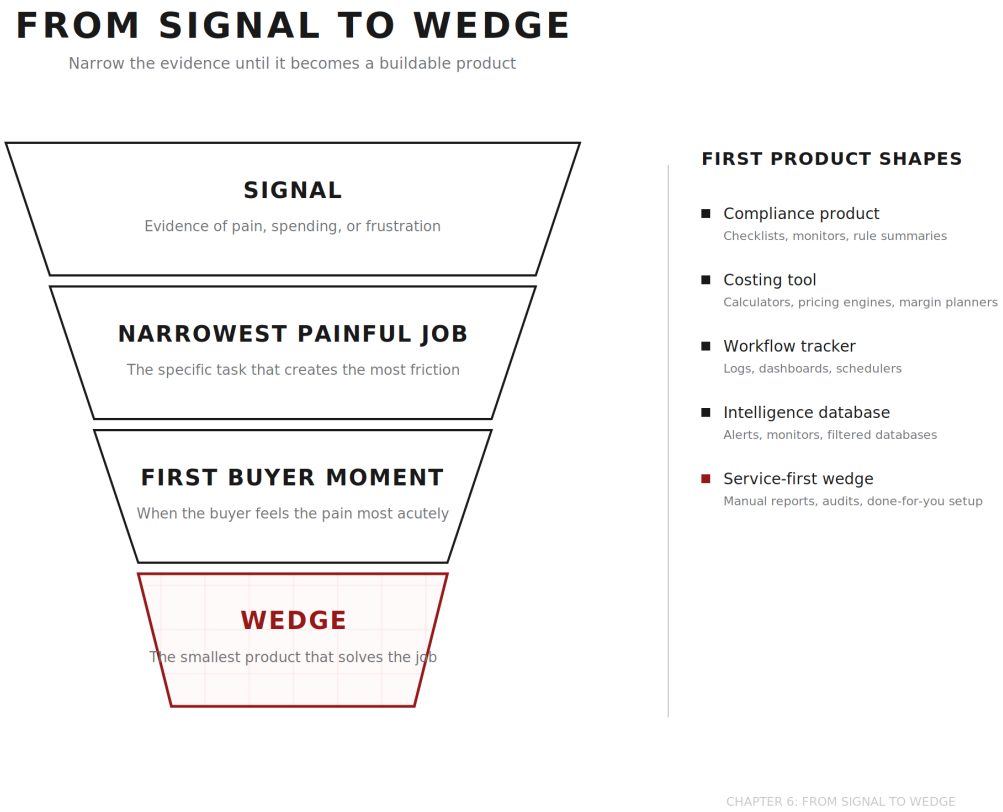
If the best argument for the idea is that the stack is interesting, the market is probably not the real driver.

1.6. From Signal to Wedge

A pain signal is not a product. It is evidence.

Signal → narrowest painful job → first buyer moment → smallest product shape.

Your job is to turn that evidence into a wedge narrow enough to validate and strong enough to matter.

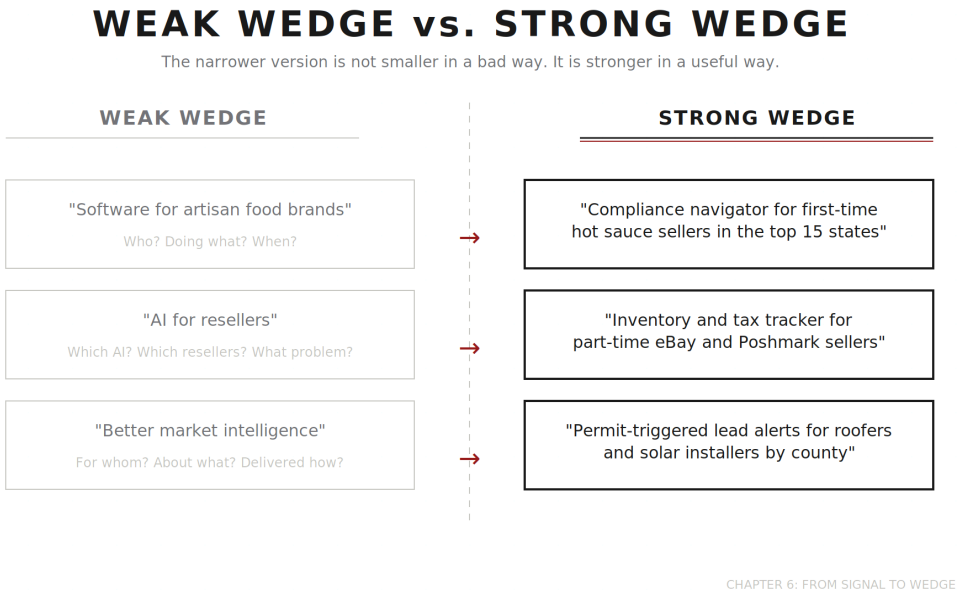


Common first product shapes include compliance products (checklists, monitors, rule summaries), costing tools (calculators, pricing engines,

margin planners), workflow trackers (logs, dashboards, schedulers), searchable intelligence databases (alerts, monitors, filtered databases), and service-first wedges (manual reports, audits, done-for-you setup).

1.6.1. The Wedge Test

A good wedge usually has one obvious buyer, one painful job, one trigger moment, one clear promise, and one believable first version.



The narrower version is not smaller in a bad way. It is stronger in a useful way.

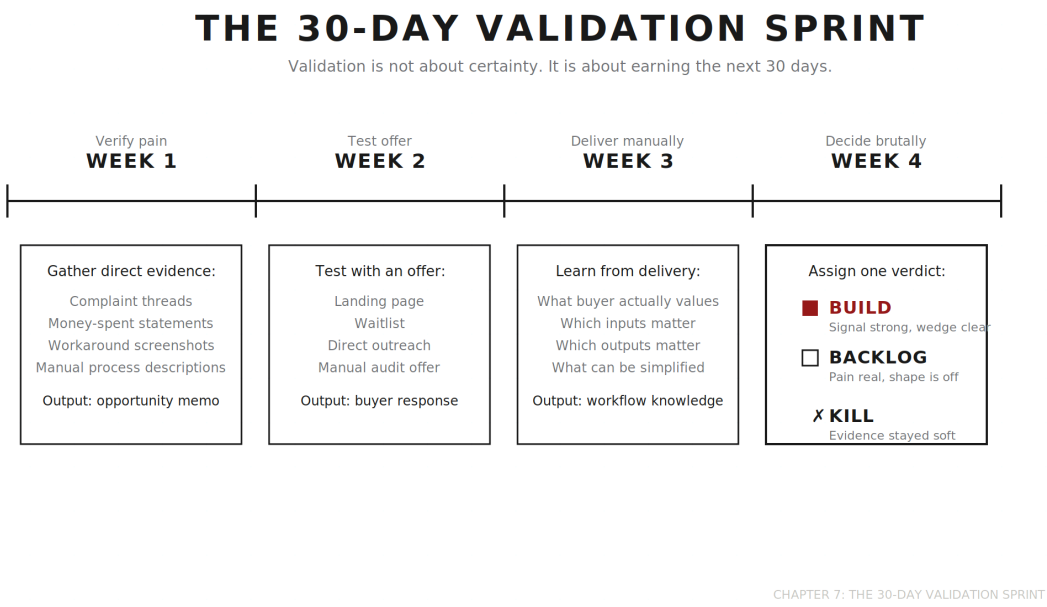
Validation and Execution

This part turns the framework into action: how to validate quickly, sell before overbuilding, and judge what deserves the next 30 days.

1.7. The 30-Day Validation Sprint

Validation is not about certainty. It is about earning the next 30 days.

Four questions: Is the pain real? Is the buyer reachable? Will anyone pay? Is the wedge small enough to ship?



CHAPTER 7: THE 30-DAY VALIDATION SPRINT

1.7.1. Week 1: Verify the Pain

Gather direct evidence: complaint threads, money-spent statements, workaround screenshots, manual process descriptions, compliance confusion, repeated buying triggers. Then write a one-page opportunity memo.

1.7.2. Week 2: Test the Offer

You do not need a product yet. You need an offer. Test it with a landing page, waitlist, direct outreach, a manual audit offer, or a concierge service.

1.7.3. Week 3: Deliver Manually

This is where many founders cheat. They collect emails and call that validation. Usually it is not.

If no one wants the manual result, software will not save you.

A concrete example: if you are testing permit-triggered lead alerts, you do not need a dashboard first. You can manually pull filings for one county, clean them in a spreadsheet, and send a weekly alert email to three contractors. If they reply, ask questions, or ask you to keep sending it, you have signal.

1.7.4. Week 4: Decide Brutally

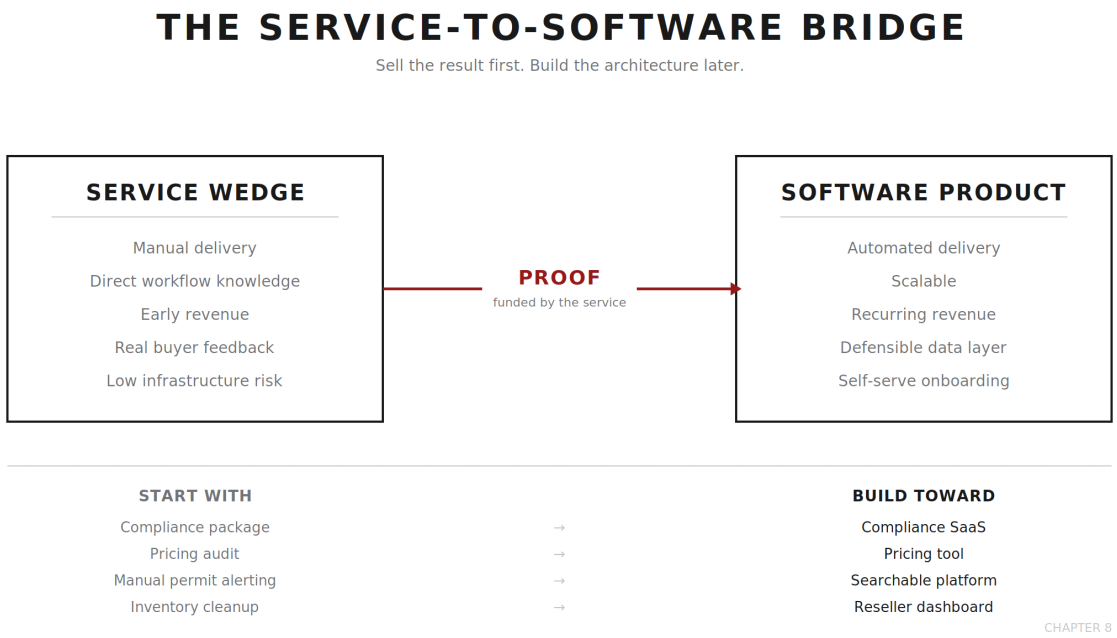
Then assign one verdict: **BUILD** if the signal is strong and the wedge is clear. **BACKLOG** if the pain is real but timing, access, or shape is off. **KILL** if the evidence stayed soft or the build path is too heavy.

1.8. First Customer Before Full Product

Most solo founders overbuild because software feels like progress. Often it is avoidance.

Sell the result. The buyer cares about the outcome, not the architecture.

The fastest way to learn whether a wedge matters is to get a first customer before the product is fully formed. One of the cleanest moves for solo builders is to start with a service-shaped wedge before converting it into software.



This does three things: gets you closer to money faster, gives you direct workflow knowledge, and funds the build instead of waiting for the build to fund itself.

1.9. Framework in Action

The point of this section is not volume. It is proof.

These are not recommendations. They are teaching examples showing what strong opportunity structure looks like.

These are the kinds of wedges the framework tends to surface when the filters are doing their job.

SCORED OPPORTUNITY CARD

How to read an opportunity through the six filters

SAUCESTACK EXAMPLE CARD

BUYER
First-time hot sauce entrepreneurs

WEDGE
Compliance navigator + label requirement checker

PAIN
State-by-state cottage food confusion, labeling requirements, co-packer uncertainty

FILTER SCORES

Buyer Clarity	5	Can name exact buyer
Pain Severity	4	Delays launch, costs money
Monetization Clarity	4	Checklist upsell, service layer
Build Feasibility	5	Starts as checklist, no infrastructure
Defensibility	3	Moderate: content can be copied
Distribution Realism	4	SEO + communities reachable
TOTAL	**25 / 30**	**RDICT: BUILD CANDIDA**

CHAPTER 9: FRAMEWORK IN ACTION

1.9.1. 1. SauceStack

Buyer: First-time hot sauce entrepreneurs. **Pain:** State-by-state cottage food confusion, labeling requirements, and co-packer uncertainty. **Wedge:** Compliance navigator plus label requirement checker.

SauceStack scores 25/30. Buyer clarity and build feasibility are perfect 5s. Defensibility is the weak point at 3.

Why it works: Clear buyer, urgent trigger, and obvious willingness to pay because mistakes delay launch. **What the framework sees:** Visible regulatory pain, a strong first-customer moment, and a narrow entry wedge that can begin as a checklist before software deepens.

1.9.2. 2. FlipTrack

Buyer: Part-time and full-time resellers. **Pain:** Inventory chaos, tax-season panic, unclear profit by item or platform. **Wedge:** Cross-platform inventory plus tax-ready export.

Why it works: Repetitive pain, recurring use, measurable value. **What the framework sees:** Ugly repeated workflow, obvious cost of mistakes, and a manual bridge that can start with cleanup plus reporting.

1.9.3. 3. Building Permit Cross-Sell Intelligence

Buyer: Roofers, solar installers, remodelers, and adjacent home-service operators. **Pain:** Weak visibility into new jobs at the right time. **Wedge:** Permit-triggered lead alerts by trade and geography.

Permit data is the classic public-data arbitrage play: the data exists, the interface is terrible, and the timing value is obvious to the buyer.

Why it works: The timing value is easy to explain and easy to sell. **What the framework sees:** Public-data arbitrage plus strong distribution realism.

1.9.4. 4. Federal Contracting Opportunity Engine

Buyer: Government contractors, consultants, and capture teams. **Pain:** Incumbent tools are expensive and government interfaces are painful. **Wedge:** Better filtering, search, and alerting around solicitations.

Why it works: Clear spend, ugly existing experience, high value per win.

1.9.5. 5. CraftLedger

Buyer: Leather crafters and commission-based makers. **Pain:** Underpricing labor, weak quoting discipline, poor materials tracking. **Wedge:** Project costing and pricing calculator.

Why it works: Small pricing improvements create immediate ROI. **What the framework sees:** A narrow buyer, recurring pain, and a wedge that can be sold as a tool, worksheet, or service before it becomes software.

The common pattern should be obvious by now. The strongest opportunities are usually not the most glamorous. They are the ones where pain is visible, the trigger is real, and the first version can solve one painful job without pretending to be a whole company on day one.

1.10. What to Build Next

The final decision is not, "What sounds best?"

It is, "What deserves the next 30 days more than the alternatives?"

Choose the opportunity that best combines strongest score, clearest wedge, fastest path to proof, best fit with your actual strengths, and lowest risk of dragging you into a swamp before first revenue.

The best next build is often not the biggest market. It is the cleanest entry point.

Parallel idea pursuit feels productive. Usually it is focus leakage.

Do not let two promising ideas advance at the same time unless one is clearly a sidecar asset.

Build one. Backlog one. Kill the rest.

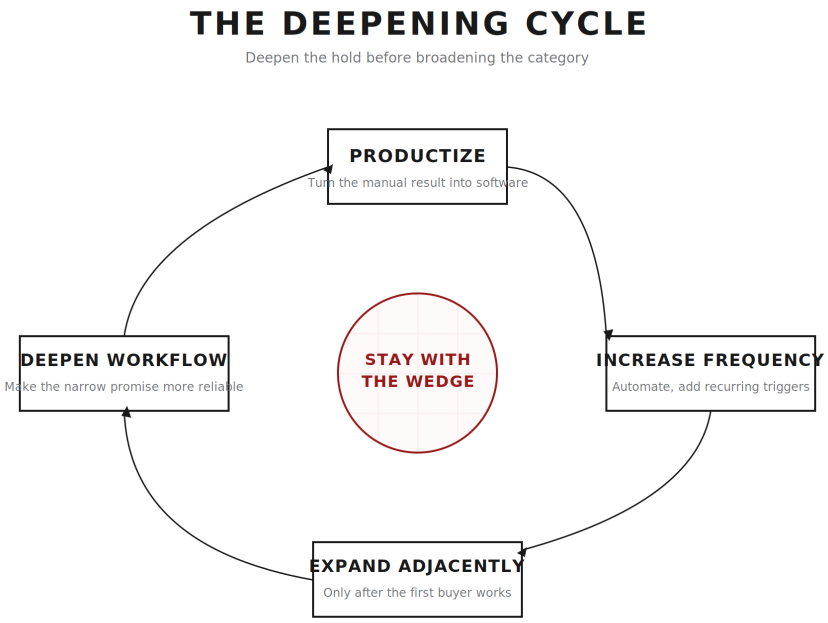
That is enough.

1.11. What Comes After the Wedge

If the wedge works, the next job is not to explode the scope. It is to deepen the proof.

The four post-wedge moves: productize, increase frequency, expand adjacently, deepen workflow.

That usually means one of four moves: productize the manual result, increase frequency or automation, expand to adjacent buyers only after the first buyer is working, and deepen the workflow before broadening the category.



The next phase should feel like deepening a hold, not starting over with a bigger fantasy.

CHAPTER 11: WHAT COMES AFTER THE WEDGE

This is where a lot of builders relapse. They survive the idea trap, prove a narrow wedge, then immediately try to turn it into a broad platform because the early signal feels like permission to expand.

It usually is not.

The wedge earned the next phase because it was specific. Keep respecting that. Stay with the buyer. Stay with the trigger. Stay with the painful job. Make the recurring version better before you start imagining the category diagram.

The next phase should feel like deepening a hold, not starting over with a bigger fantasy.

Conclusion

This framework sounds ruthless because it is.

That is also why it is useful.

The hard part is not understanding the filters. The hard part is using them against ideas you actually like. Killing weak ideas is emotionally expensive because weak ideas are rarely worthless. They are usually just mistimed, under-shaped, under-validated, or not strong enough to deserve the next few months of your life.

You are not usually cutting obvious trash. You are cutting things that could maybe become something later.

That is what makes discipline difficult. You are not usually cutting obvious trash. You are cutting things that could maybe become something later. But “maybe later” is exactly how builders get trapped in soft conviction and scattered attention.

This book is not here to make you feel like you have more options.

It is here to make you choose better.

You do not need a hundred ideas. You need one strong wedge, one real buyer, one believable path to money, and the discipline to ignore everything else until the market answers you.

That is enough to build something real.

2. Appendices

I Pattern Library

These are shorter opportunity patterns worth studying. They are pattern exercises, not the main proof section.

2.1.0.1. HiveStack

Buyer: Sideline beekeepers. **Pain:** Colony loss, treatment timing, weak profitability tracking. **Wedge:** Hive log plus treatment scheduler plus simple economics tracker.

2.1.0.2. Liquor License Opening Alerts

Buyer: Restaurant vendors, equipment suppliers, POS sellers. **Pain:** Missing the earliest buying window. **Wedge:** License approval alert feed.

2.1.0.3. CFPB Complaint Intelligence

Buyer: Consumers, analysts, financial marketers. **Pain:** Complaint data exists but is hard to interpret. **Wedge:** Searchable complaint trends and reputation monitoring.

2.1.0.4. Restaurant Inspection Crisis Alerts

Buyer: Restaurant operators and reputation managers. **Pain:** Inspection failures damage revenue fast. **Wedge:** Alerting plus rapid-response checklist.

2.1.0.5. Contractor License Violation Monitor

Buyer: Contractors, insurers, sureties. **Pain:** Licensing lapses are hard to track across jurisdictions. **Wedge:** Monitoring plus alerting plus referral hooks.

2.1.0.6. Small-Batch Production Tracker

Buyer: Artisan food and handmade-goods producers. **Pain:** Spreadsheet-heavy batch logs, weak costing. **Wedge:** Batch management for one production type first.

2.1.0.7. UCC Filing Lead Intelligence

Buyer: Insurers, lenders, equipment vendors. **Pain:** Weak visibility into newly financed businesses. **Wedge:** Financing event trigger intelligence.

2.1.0.8. Compliance Launch Planner

Buyer: Niche operators moving from hobby to business. **Pain:** No structured path across entity setup, compliance, and first sales. **Wedge:** Vertical-specific launch planner with templates.

II Opportunity Scorecard

Rate each idea from **1 to 5** on: buyer clarity, pain severity, monetization clarity, build feasibility, defensibility, and distribution realism.

Interpretation: 26–30 = strong build candidate. 21–25 = good candidate, pressure-test first. 16–20 = backlog or tighten the wedge. 15 and below = kill unless new evidence appears.

III Opportunity Worksheet

For every serious opportunity, fill this out before building:

Working name .....

Buyer .....

Pain .....

Current workaround .....

Trigger moment .....

Wedge .....

Why now .....

Path to money .....

Fastest way to test .....

Main risk .....

Build / Backlog / Kill .....

If you cannot complete this in plain English, the idea is still too fuzzy.